EF

what is ORM?

Orm is a Object Relational Mapping. It is a technique that allows you to query and manipulate data from a database using an object-oriented paradigm.

What is EF Core?

EF Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server, SQLite, Azure Cosmos DB, MySQL, PostgreSQL, and other databases through a provider plugin API.

What is the difference between EF and EF Core?

EF Core is a lightweight, extensible, and cross-platform version of Entity Framework. It has better performance, new features, and is continuously updated.

What are the advantages of using Entity Framework?

- Faster development
- Less code
- Easy to maintain
- Supports LINQ
- Cross-platform
- Database-agnostic

What are the disadvantages of using Entity Framework?

- Performance overhead
- Limited control over SQL queries
- Learning curve
- Not suitable for complex queries
- Database-specific features may not be supported

How to install Entity Framework?

You can install Entity Framework using NuGet Package Manager in Visual Studio. Run the following command in the Package Manager Console:

```
Install-Package EntityFramework
```

How to create a new Entity Framework context?

You can create a new Entity Framework context by inheriting from the `DbContext` class and defining the database tables as properties. Here is an example:

```
public class MyDbContext : DbContext
{
    public DbSet<Customer> Customers { get; set; }
    public DbSet<Order> Orders { get; set; }
}
```

How to do datafirst approach in EF?

Entity Framework supports the Database-First approach, where you can generate entity classes and a context class from an existing database.
Using The CLI:

```
Scaffold-DbContext
"Server=.;Database=MyDatabase;Trusted_Connection=True;"
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models
```

Using Power Tools:

- Right-click on the project
- Select "Entity Framework" -> "Reverse Engineer Code First"
- Follow the wizard to generate the entity classes and context class

Immediate vs Deferred Execution in EF?

Immediate Execution: Queries that are executed immediately when they are defined. For example, when you use methods like `ToList()`, `First()`, `Single()`, etc.
Deferred Execution: Queries that are not executed immediately but are executed when

the query results are enumerated. For example, when you use methods like `Where()`, `Select()`, `OrderBy()`, etc.

How to use LINQ with Entity Framework?

You can use LINQ (Language Integrated Query) with Entity Framework to query data from the database. Here is an example:

```
using (var context = new MyDbContext())
{
    var customers = context.Customers
        .Where(c => c.City == "London")
        .OrderBy(c => c.LastName)
        .ToList();
}
```

How to view the state of an entity in Entity Framework?

You can view the state of an entity in Entity Framework using the `Entry()` method of the `DbContext` class. Here is an example:

```
using (var context = new MyDbContext())
{
    var customer = context.Customers.Find(1);
    var entry = context.Entry(customer);
    var state = entry.State;
}
```

or

```
using (var context = new MyDbContext())
{
    var customer = context.Customers.Find(1);
    var state = context.Entry<Customers>(customer).State;
}
```

what are the different states of an entity in Entity Framework?

There are four different states of an entity in Entity Framework:

- Added: The entity is being tracked by the context and will be inserted into the database when `SaveChanges()` is called.
- Modified: The entity is being tracked by the context and its properties have been modified. It will be updated in the database when `SaveChanges()` is called.
- Deleted: The entity is being tracked by the context and will be deleted from the database when `SaveChanges()` is called.
- Unchanged: The entity is being tracked by the context and its properties have not been modified since it was queried from the database.

How to access the original and current values of an entity in Entity Framework?

You can access the original and current values of an entity in Entity Framework using the `OriginalValues` and `CurrentValues` properties of the `EntityEntry` class. Here is an example:

```
using (var context = new MyDbContext())
{
    var customer = context.Customers.Find(1);
    var entry = context.Entry(customer);
    var originalValues = entry.OriginalValues;
    var currentValues = entry.CurrentValues;

    var originalCity = originalValues[nameof(Customer.City)];
    var currentCity = currentValues[nameof(Customer.City)];
}
```

using Partial Classes in Database First Approach in EF Core

Partial classes are used to split the definition of a class across multiple files. You can use partial classes to extend the generated entity classes in the Database-First approach in EF Core.
Use them to add custom properties or methods or to the generated entity classes without modifying the generated code.
Here is an example of using partial classes with the Database-First approach in EF Core:

```
// Generated entity class
public partial class Customer
{
    public int Id { get; set; }
```

```
    public string Name { get; set; }
}

// Partial class in a separate file
public partial class Customer
{
    public string FullName => $"{Id} - {Name}";
}
```

ModelMetadataType Attribute in EF Core

The `ModelMetadataType` attribute is used to associate a metadata class with an entity class in EF Core. The metadata class contains additional metadata information about the entity class, such as display names, validation rules, etc.
Here is an example of using the `ModelMetadataType` attribute in EF Core:

```
[ModelMetadataType(typeof(CustomerMetadata))]
public partial class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
}

public class CustomerMetadata
{
    [Display(Name = "Customer ID")]
    public int Id { get; set; }

    [Display(Name = "Customer Name")]
    public string Name { get; set; }
}
```

Code First Approach in EF Core

The Code-First approach in EF Core allows you to define your entity classes and their relationships in code, and then generate the database schema from the code. You can use attributes or fluent API to configure the database schema.
Here is an example of using the Code-First approach in EF Core:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

```
public class Order
{
    public int Id { get; set; }
    public int CustomerId { get; set; }
    public Customer Customer { get; set; }
}

public class MyDbContext : DbContext
{
    public DbSet<Customer> Customers { get; set; }
    public DbSet<Order> Orders { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
    {

optionsBuilder.UseSqlServer("Server=.;Database=MyDatabase;Trusted_Connection
=True;");
    }
}
```

```
add-migration InitialCreate
update-database
```

Remove-Migration in EF Core

The `Remove-Migration` command in EF Core is used to remove the last migration from the project. This command is useful when you want to undo the changes made by the last migration.
Here is an example of using the `Remove-Migration` command in EF Core:

```
Remove-Migration
```

How to seed data in EF Core?

You can seed data in EF Core by overriding the `OnModelCreating` method of the `DbContext` class and using the `HasData` method to add seed data to the database.
Here is an example of seeding data in EF Core:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
```

```
    modelBuilder.Entity<Customer>().HasData(
        new Customer { Id = 1, Name = "Alice" },
        new Customer { Id = 2, Name = "Bob" }
    );
}
```

Convention in EF Core

Conventions in EF Core are a set of rules that define how the entity classes are mapped to the database schema. EF Core uses conventions to infer the database schema from the entity classes without explicit configuration.
Here are some common conventions in EF Core:

- Primary Key Convention: EF Core infers the primary key property based on the property name or the `Id` property.
- Foreign Key Convention: EF Core infers the foreign key property based on the navigation property name or the `NavigationPropertyNameId` property.
- Table Name Convention: EF Core infers the table name based on the entity class name.
- Column Name Convention: EF Core infers the column name based on the property name.
- Relationship Convention: EF Core infers the relationship between entities based on the navigation properties.

Data Annotations in EF Core

Data Annotations in EF Core are attributes that you can apply to entity classes and properties to configure the database schema. Data Annotations provide a declarative way to configure the database schema without using fluent API.
Here are some common Data Annotations in EF Core:

- `[Key]` : Specifies the primary key property.
- `[ForeignKey]` : Specifies the foreign key property.
- `[Table]` : Specifies the table name.
- `[Column]` : Specifies the column name.
- `[Required]` : Specifies that the property is required.
- `[MaxLength]` : Specifies the maximum length of a string property.
- `[NotMapped]` : Specifies that the property is not mapped to the database.
- `[DatabaseGenerated]` : Specifies how the value of a property is generated.

Fluent API in EF Core

Fluent API in EF Core is a set of methods that you can use to configure the database schema. Fluent API provides a programmatic way to configure the database schema without using Data Annotations.
Here are some common Fluent API methods in EF Core:

- `HasKey`: Specifies the primary key property.
- `HasOne`: Specifies the navigation property in a one-to-one relationship.
- `HasMany`: Specifies the navigation property in a one-to-many relationship.
- `HasForeignKey`: Specifies the foreign key property.
- `ToTable`: Specifies the table name.
- `HasColumnName`: Specifies the column name.
- `IsRequired`: Specifies that the property is required.
- `HasMaxLength`: Specifies the maximum length of a string property.
- `Ignore`: Specifies that the property is not mapped to the database.
- `ValueGeneratedOnAdd`: Specifies that the value of a property is generated when the entity is added to the database.

How to configure relationships in EF Core?

You can configure relationships in EF Core using Data Annotations or Fluent API. You can specify the relationship type, foreign key property, navigation properties, and other configuration options.
Here is an example of configuring a one-to-many relationship using Data Annotations:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public List<Order> Orders { get; set; }
}

public class Order
{
    public int Id { get; set; }
    public int CustomerId { get; set; }
    public Customer Customer { get; set; }
}


public class MyDbContext : DbContext
{
    public DbSet<Customer> Customers { get; set; }
```

```
    public DbSet<Order> Orders { get; set; }
}
```

or using Fluent API:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .HasMany(c => c.Orders)
        .WithOne(o => o.Customer)
        .HasForeignKey(o => o.CustomerId);
}
```

or In Reverse

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>()
        .HasOne(o => o.Customer)
        .WithMany(c => c.Orders)
        .HasForeignKey(o => o.CustomerId);
}
```

Mapping many-to-many relationships in EF Core

You can map many-to-many relationships in EF Core using a join table. You can create a join entity that represents the relationship between the two entities and configure the relationships using Data Annotations or Fluent API.
Here is an example of mapping a many-to-many relationship using a join entity and Fluent API:

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public List<StudentCourse> StudentCourses { get; set; }
}

public class Course
{
    public int Id { get; set; }
    public string Name { get; set; }
    public List<StudentCourse> StudentCourses { get; set; }
}
```

```
public class StudentCourse
{
    public int StudentId { get; set; }
    public Student Student { get; set; }
    public int CourseId { get; set; }
    public Course Course { get; set; }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<StudentCourse>()
        .HasKey(sc => new { sc.StudentId, sc.CourseId });

    modelBuilder.Entity<StudentCourse>()
        .HasOne(sc => sc.Student)
        .WithMany(s => s.StudentCourses)
        .HasForeignKey(sc => sc.StudentId);

    modelBuilder.Entity<StudentCourse>()
        .HasOne(sc => sc.Course)
        .WithMany(c => c.StudentCourses)
        .HasForeignKey(sc => sc.CourseId);
}
```

This Type Of Mapping is Only Required When You Need To Store Additional Information In The Join Table

How to Map One-to-One Relationships in EF Core

You can map one-to-one relationships in EF Core using Data Annotations or Fluent API. You can specify the navigation properties, foreign key properties, and other configuration options.
Here is an example of mapping a one-to-one relationship using Fluent Api:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public CustomerProfile Profile { get; set; }
}

public class CustomerProfile
{
    public int Id { get; set; }
    public string Address { get; set; }
```

```
    public int CustomerId { get; set; }
    public Customer Customer { get; set; }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .HasOne(c => c.Profile)
        .WithOne(p => p.Customer)
        .HasForeignKey<CustomerProfile>(p => p.CustomerId);
}
```

or using Data Annotations:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public CustomerProfile Profile { get; set; }
}

public class CustomerProfile
{
    public int Id { get; set; }
    public string Address { get; set; }
    public int CustomerId { get; set; }
    public Customer Customer { get; set; }
}
```

The ForeignKey Property Should Be Placed In The Dependent Entity (Not Required In The Principal Entity)

Self-Referencing Relationships in EF Core

You can create self-referencing relationships in EF Core by defining a navigation property that points to the same entity type. You can configure the relationship using Data Annotations or Fluent API.
Here is an example of a self-referencing relationship using Fluent API:

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int? ManagerId { get; set; } // Self-referencing foreign key //
nullable because the manager is optional for the manager himself
```

```
    public Employee Manager { get; set; }
    public List<Employee> Subordinates { get; set; }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Employee>()
        .HasOne(e => e.Manager)
        .WithMany(e => e.Subordinates)
        .HasForeignKey(e => e.ManagerId);
}
```

Poco Classes in EF Core

POCO (Plain Old CLR Object) classes in EF Core are simple classes that represent the entities in your application. POCOs are simple class objects that do not inherit features from any framework-specific base classes or interfaces.
Here is an example of a POCO class in EF Core:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

POCO classes don't have any data annotations.

How to configure inheritance in EF Core?

You can configure inheritance in EF Core using the Table Per Hierarchy (TPH), Table Per Type (TPT), or Table Per Concrete Type (TPC) strategies. You can use Data Annotations or Fluent API to configure the inheritance mapping.
Here is an example of configuring inheritance using the Table Per Hierarchy (TPH) strategy:

```
public class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
}

public class Employee : Person
{
```

```
    public decimal Salary { get; set; }
}

public class Customer : Person
{
    public string Email { get; set; }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Person>()
        .HasDiscriminator<string>("PersonType") // overriding the default
discriminator column name "Discriminator" to "PersonType"
        .HasValue<Person>("Person")
        .HasValue<Employee>("Employee")
        .HasValue<Customer>("Customer");
}
```

Advantages of TPH

- Single table for all entities
- Simple schema
- Easy to query
- Easy to maintain

  [!warn] Disadvantages of TPH
- Null columns for non-matching properties
- Performance overhead for large tables
- Limited flexibility

TPT Inheritance in EF Core

The Table Per Type (TPT) inheritance strategy in EF Core creates a separate table for each entity type in the inheritance hierarchy. Each table contains the properties specific to that entity type.
Here is an example of configuring inheritance using the Table Per Type (TPT) strategy:

```
public class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
}

public class Employee : Person
```

```
{
    public decimal Salary { get; set; }
}

public class Customer : Person
{
    public string Email { get; set; }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Person>().UseTptMappingStrategy();
}
```

Advantages of TPT

- Separate tables for each entity
- No null columns
- More flexibility

[!warn] Disadvantages of TPT

- More complex schema
- More joins required for queries
- Harder to maintain

TPC Inheritance in EF Core

The Table Per Concrete Type (TPC) inheritance strategy in EF Core creates a separate table for each concrete entity type in the inheritance hierarchy. Each table contains the properties of the concrete entity type.
Here is an example of configuring inheritance using the Table Per Concrete Type (TPC) strategy:

```
public class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
}

public class Employee : Person
{
    public decimal Salary { get; set; }
}
```

```
public class Customer : Person
{
    public string Email { get; set; }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Person>().UseTpcMappingStrategy();

}
```

EnsureCreated vs EnsureDeleted in EF Core

The `EnsureCreated` method in EF Core is used to create the database if it does not exist. It creates the database schema based on the entity classes and relationships defined in the `DbContext`.
The `EnsureDeleted` method in EF Core is used to delete the database if it exists. It deletes the database schema and all the data in the database.
Here is an example of using `EnsureCreated` and `EnsureDeleted` methods in EF Core:

```
using (var context = new MyDbContext())
{
    context.Database.EnsureCreated(); // Create the database if it does not
exist
    context.Database.EnsureDeleted(); // Delete the database if it exists
}
```

How to untrack an entity in EF Core?

You can untrack an entity in EF Core by using the `Detach` method of the `DbContext` class. The `Detach` method removes the entity from the change tracker, so changes to the entity will not be tracked.
Here is an example of untracking an entity in EF Core:

```
using (var context = new MyDbContext())
{
    var customer = context.Customers.Find(1);
    context.Entry(customer).State = EntityState.Detached; // Untrack the
entity
}
```

AsNoTracking in EF Core

The `AsNoTracking` method in EF Core is used to disable change tracking for a query. When you use `AsNoTracking`, EF Core does not track the entities returned by the query, which can improve performance for read-only queries.
Here is an example of using `AsNoTracking` in EF Core:

```
using (var context = new MyDbContext())
{
    var customers = context.Customers.AsNoTracking().ToList(); // Disable
change tracking
}
```

to stop tracking all the objects from the database in EF Core using UseQueryTrackingBehavior

You can stop tracking all the objects from the database in EF Core using the `UseQueryTrackingBehavior` method. The `UseQueryTrackingBehavior` method allows you to configure the query tracking behavior for all queries in the `DbContext`.
Here is an example of using `UseQueryTrackingBehavior` in EF Core:

```
protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
{

optionsBuilder.UseSqlServer("Server=.;Database=MyDatabase;Trusted_Connection
=True;")
        .UseQueryTrackingBehavior(QueryTrackingBehavior.NoTracking);
}
```

Server Side vs Client Side Evaluation in EF Core

Server-side evaluation in EF Core means that the query is translated to SQL and executed on the database server. Server-side evaluation is more efficient because it reduces the amount of data transferred between the database server and the application.
Client-side evaluation in EF Core means that the query is executed on the client-side after the data is retrieved from the database. Client-side evaluation can be less efficient because it requires loading all the data into memory before executing the query.
Here is an example of server-side evaluation and client-side evaluation in EF Core:

```
using (var context = new MyDbContext())
{
    var customers = context.Customers.Where(c => c.City ==
"London").ToList(); // Server-side evaluation
    var customers = context.Customers.ToList().Where(c => c.City ==
"London").ToList(); // Client-side evaluation
}
```

Eager Loading vs Lazy Loading in EF Core

Eager loading in EF Core means that related entities are loaded along with the main entity in a single query. Eager loading can reduce the number of queries executed against the database but can result in loading more data than necessary.
Lazy loading in EF Core means that related entities are loaded on-demand when they are accessed for the first time. Lazy loading can reduce the amount of data loaded initially but can result in additional queries being executed against the database.
Here is an example of eager loading and lazy loading in EF Core:

```
using (var context = new MyDbContext())
{
    var customers = context.Customers.Include(c => c.Orders).ToList(); //
Eager loading
    var customer = context.Customers.Find(1);
    var orders = customer.Orders.ToList(); // Lazy loading
}
```

How to Make Lazt Loading Work in EF Core

To make lazy loading work in EF Core, you need to install the `Microsoft.EntityFrameworkCore.Proxies` package and enable lazy loading in the `DbContext` class. You can enable lazy loading by setting the `UseLazyLoadingProxies` option to `true` in the `OnConfiguring` method.
Here is an example of enabling lazy loading in EF Core:

```
protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
{

optionsBuilder.UseSqlServer("Server=.;Database=MyDatabase;Trusted_Connection
=True;")
        .UseLazyLoadingProxies();
}
```

Navigation properties MUST be virtual with UseLazyLoadingProxies

if we have a navigation property without virtual keyword we will get an error
as proxy class will not be able to override the navigation property

Refrence Method in EF Core

The `Reference` method in EF Core is used to include a related entity in the query result. The `Reference` method is used to load a single related entity in a one-to-one or many-to-one relationship.

Collection Method in EF Core

The `Collection` method in EF Core is used to include a collection of related entities in the query result. The `Collection` method is used to load multiple related entities in a one-to-many or many-to-many relationship.

[Ayman] Eagar loading vs Lazy loading

in web applications we use Eagar loading to get all the data in one request
in desktop applications we use Lazy loading to get the data when it is needed

[Ayman] doing configuration for all entities in OnModelCreating method is not a good practice

better to create a configuration file for each entity
StudentConfiguration class
inherit from IEntityTypeConfiguration interface
override Configure method

```
public class StudentConfiguration : IEntityTypeConfiguration<Student>
{
    public void Configure(EntityTypeBuilder<Student> builder)
    {
        builder.HasKey(s => s.Id);
        builder.Property(s => s.Name).IsRequired();
    }
}
```

then in the OnModelCreating method

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfiguration(new StudentConfiguration());
}
```

ApplyingConfigurationFromAssembly in EF Core

The `ApplyConfigurationFromAssembly` method in EF Core is used to apply entity configurations from an assembly. The `ApplyConfigurationFromAssembly` method scans the specified assembly for classes that implement the `IEntityTypeConfiguration` interface and applies the configurations to the `ModelBuilder`.
Here is an example of using `ApplyConfigurationFromAssembly` in EF Core:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{

modelBuilder.ApplyConfigurationFromAssembly(Assembly.GetExecutingAssembly())
;
}
```